Chapter 12. Support Vector Machines

Lecture 9 (hyperplanes, maximal margin classifier, support vector classifier, kernels, multiclass SVM, SVM vs logistic regression)

Contents

1 Separating Hyperplanes **2**
1.1 Motivation . . . 2
1.2 Hyperplanes . . . 2

2 The Maximal Margin Classifier **2**
2.1 Non-separable and noisy data . . . 3

3 The Support Vector Classifier **3**

4 Support Vector Machines **4**
4.1 Feature expansion . . . 4
4.2 Inner products and kernels . . . 4

5 SVMs With More Than Two Classes **5**

6 SVM versus Logistic Regression **5**

1 Separating Hyperplanes

1.1 Motivation

In a two-class problem with predictors X_1, X_2, suppose the classes are *linearly separable*: a straight line puts all points of one class on one side and all of the other class on the other. A natural approach is to find the line that gives the *biggest separation*, the points being as far from it as possible. This is the basic idea of support vector classifiers. For the general two-class problem we try to find a plane that separates the classes in feature space; if we cannot, we get creative in two ways: we soften what "separates" means, and we enrich and enlarge the feature space so that separation becomes possible.

1.2 Hyperplanes

A hyperplane in p dimensions is a flat affine subspace of dimension $p-1$:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0;$$

in $p = 2$ dimensions it is a line. With $f(X) = \beta_0 + \beta^\top X$, $f(X) > 0$ on one side and $f(X) < 0$ on the other. Coding $y_i = +1$ for one class and $y_i = -1$ for the other, $f(X) = 0$ is a **separating hyperplane** if $y_i f(x_i) > 0$ for all i; then classify x by $\operatorname{sign} f(x)$, and $|f(x)|$ measures confidence.

Theorem: distance to a hyperplane

The signed distance of x to $\{f = 0\}$ is $f(x)/\|\beta\|$; if $\|\beta\| = 1$, $f(x)$ itself is the signed distance.

Proof. β is normal to the hyperplane. For a point x_0 on it, the distance is the length of the projection of $x - x_0$ on $\beta/\|\beta\|$: $\beta^\top(x - x_0)/\|\beta\| = (f(x) - f(x_0))/\|\beta\| = f(x)/\|\beta\|$. ■

2 The Maximal Margin Classifier

If the data are separable there are infinitely many separating hyperplanes. The **margin** M is the minimum perpendicular distance from the observations to the hyperplane; the **optimal separating hyperplane** maximises it:

$$\max_{\beta_0,\beta} M \quad \text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1, \qquad y_i(\beta_0 + \beta^\top x_i) \geq M \quad \forall i.$$

Theorem: equivalent convex problem

The problem is equivalent to

$$\min_{\beta_0,\beta} \frac{1}{2}\|\beta\|^2 \quad \text{subject to} \quad y_i(\beta_0 + \beta^\top x_i) \geq 1 \quad \forall i,$$

with margin $M = 1/\|\beta\|$.

Proof. Drop the norm constraint and write the condition as $y_i(\beta_0 + \beta^\top x_i)/\|\beta\| \geq M$ (distances, by the previous theorem). The scale of (β_0, β) is arbitrary, so fix it by $\|\beta\| = 1/M$; then the constraints become $y_i(\beta_0 + \beta^\top x_i) \geq 1$ and maximising $M = 1/\|\beta\|$ is minimising $\|\beta\|^2/2$, a convex quadratic program. ■

The solution depends only on the points lying on the margin, the **support vectors**: moving any other point (without crossing the margin) does not change the hyperplane.

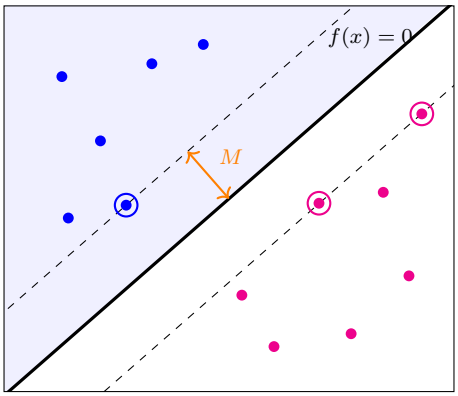


Figure 1: Maximal margin hyperplane (solid) and margins (dashed) for separable data. The circled points lie on the margins: they are the support vectors, and only they determine the hyperplane. M is the distance from the hyperplane to the margin (schematic).

2.1 Non-separable and noisy data

Often the data are not separable by a linear boundary (usually the case unless $n < p$), and then the problem has no solution. Even when separable, data may be noisy: a single observation can move the maximal margin hyperplane dramatically and produce a tiny margin. We want greater robustness to individual observations and better classification of *most* training observations: the support vector classifier maximises a **soft margin**.

3 The Support Vector Classifier

Definition: soft-margin optimisation

$$\max_{\beta_0,\beta,\epsilon} M \quad \text{subject to} \quad \sum_j \beta_j^2 = 1, \quad y_i(\beta_0 + \beta^\top x_i) \geq M(1-\epsilon_i), \quad \epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C.$$

ϵ_i are **slack variables**; $C \geq 0$ is a tuning parameter.

The slacks allow individual observations to violate the margin: $\epsilon_i = 0$ means observation i is on the correct side of the margin; $0 < \epsilon_i \leq 1$, on the wrong side of the margin but the correct side of the hyperplane; $\epsilon_i > 1$, on the wrong side of the hyperplane (misclassified). Since $\sum \epsilon_i \leq C$, at most C training observations can be misclassified.

C is a **budget** for violations. As C increases we tolerate more violations and the margin widens; as C decreases the margin narrows; $C = 0$ gives the maximal margin classifier. C is chosen by CV and controls the bias–variance trade-off: large C means many support vectors, low variance and possibly high bias; small C the opposite. Only observations on or inside the margin (or misclassified) are support vectors and affect the solution, so the classifier is robust to points far from the boundary — unlike LDA, which uses all points.

Derivation: the equivalent penalised form and the dual

In the parametrisation of the previous section, the soft-margin problem is

$$\min_{\beta_0,\beta,\xi} \frac{1}{2}\|\beta\|^2 + \gamma \sum_i \xi_i \quad \text{s.t.} \quad y_i(\beta_0 + \beta^\top x_i) \geq 1 - \xi_i, \ \xi_i \geq 0,$$

where a *large* cost γ corresponds to a *small* budget C (software usually calls γ "C" — the

opposite role). The Lagrangian dual is

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\sum_{i,i'} \alpha_i \alpha_{i'} y_i y_{i'} \langle x_i, x_{i'} \rangle \quad \text{s.t.} \quad 0 \le \alpha_i \le \gamma, \quad \sum_i \alpha_i y_i = 0,$$

and $\hat{\beta} = \sum_i \hat{\alpha}_i y_i x_i$. By complementary slackness $\hat{\alpha}_i > 0$ only for points with $y_i f(x_i) \le 1$, the support vectors. Hence $f(x) = \hat{\beta}_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i y_i \langle x, x_i \rangle$, and the data enter only through inner products.

4 Support Vector Machines

4.1 Feature expansion

Sometimes a linear boundary won't work, whatever C. As in regression (interactions, polynomials, splines), enlarge the feature space with transformations, e.g. use $(X_1, X_2, X_1^2, X_2^2, X_1X_2)$ instead of (X_1, X_2). A linear boundary in the enlarged space,

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 = 0,$$

is nonlinear (a conic) in the original space. With cubic polynomials the boundary is $\beta_0 + \cdots + \beta_6 X_1^3 + \beta_7 X_2^3 + \beta_8 X_1 X_2^2 + \beta_9 X_1^2 X_2 = 0$. Polynomials, especially high-dimensional ones, get wild rather fast; kernels introduce nonlinearities in a more elegant and controlled way.

4.2 Inner products and kernels

The inner product is $\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}$. The linear support vector classifier can be written

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle \qquad (n+1 \text{ parameters}),$$

and estimating (β_0, α) requires only the $\binom{n}{2}$ inner products between pairs of observations. Most $\hat{\alpha}_i$ are zero:

$$f(x) = \hat{\beta}_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i \langle x, x_i \rangle,$$

where $\mathcal{S}$ is the support set, $\{i : \hat{\alpha}_i \neq 0\}$. If we can compute inner products between observations, we can fit an SV classifier; replacing $\langle x, x_i \rangle$ by a **kernel** $K(x, x_i)$ — an inner product $\langle \Phi(x), \Phi(x_i) \rangle$ in some feature space — gives the **support vector machine**

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i K(x, x_i).$$

- **Polynomial kernel** of degree d: $K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^p x_{ij} x_{i'j}\right)^d$, which computes the inner products needed for d-dimensional polynomials.
- **Radial kernel**: $K(x_i, x_{i'}) = \exp\left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2\right)$, $\gamma > 0$: an implicit, infinite-dimensional feature space. Training points far from x contribute almost nothing, so the classifier is very local; γ controls the variance by squashing most dimensions severely (large γ, more flexible).

Example: the polynomial kernel is an inner product

For $p = 2$, $d = 2$: $(1+x^\top z)^2 = 1+2x_1z_1+2x_2z_2+x_1^2z_1^2+x_2^2z_2^2+2x_1x_2z_1z_2 = \langle \Phi(x), \Phi(z)\rangle$ with $\Phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$: the six features of a quadratic expansion, computed without forming them.

Kernel versus enlarged feature space. Computationally we only need $K(x_i, x_{i'})$ for the $\binom{n}{2}$ distinct pairs, whereas an explicit enlarged space can be intractable (the number of degree-d monomials grows like p^d), and some kernels (the radial kernel) correspond to no finite set of features. A function K is a valid kernel iff every Gram matrix $(K(x_i, x_{i'}))$ is positive semidefinite (Mercer).

Heart data. On the training set, ROC curves show the SV classifier slightly better than LDA, and radial SVMs with larger γ nearly perfect — overfitting; on the 90 test observations the radial SVM with large γ is worst, while the SV classifier, LDA and the radial SVM with small γ perform similarly.

5 SVMs With More Than Two Classes

The SVM as defined works for $K = 2$. Two popular extensions:

- **One versus all (OVA)**: fit K two-class SVMs $\hat{f}_k(x)$, each class versus the rest; classify x to the class with the largest $\hat{f}_k(x)$.
- **One versus one (OVO)**: fit all $\binom{K}{2}$ pairwise classifiers $\hat{f}_{kl}(x)$; classify x to the class that wins the most pairwise competitions.

If K is not too large, use OVO (each fit uses only two classes and is balanced).

6 SVM versus Logistic Regression

Theorem: SVM as loss plus penalty

The support vector classifier with $f(X) = \beta_0 + \beta_1X_1 + \cdots + \beta_pX_p$ solves

$$\min_{\beta_0,\beta} \sum_{i=1}^{n} \underbrace{\max[0,\, 1 - y_i f(x_i)]}_{\text{hinge loss}} + \lambda \sum_{j=1}^{p} \beta_j^2,$$

with large λ corresponding to a large budget C.

Proof. In the penalised form, at the optimum each slack equals $\xi_i = \max[0, 1-y_if(x_i)]$ (the smallest feasible value). Substituting gives $\frac{1}{2}\|\beta\|^2 + \gamma \sum_i \max[0, 1 - y_if(x_i)]$; divide by γ and set $\lambda = 1/(2\gamma)$. ■

This is the "loss + penalty" form, like ridge regression. The hinge loss is exactly zero for observations on the correct side of the margin ($y_if \geq 1$), which is why only support vectors matter. It is very similar to the logistic-regression loss (negative log-likelihood) $\log(1+e^{-y_if(x_i)})$, which is never exactly zero but small for $y_if \gg 0$; hence the two often give similar results.

Which to use?

- When the classes are (nearly) separable, SVM does better than logistic regression; so does LDA.

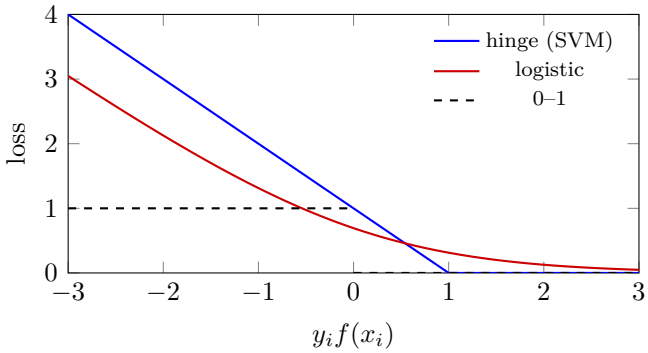


Figure 2: Hinge loss $\max(0, 1 - yf)$ is exactly zero once a point is beyond the margin ($yf \geq 1$); the logistic loss $\log(1 + e^{-yf})$ is a smooth version that never reaches zero. Both are convex surrogates of the 0–1 loss.

- When not, logistic regression (with a ridge penalty) and SVM are very similar.
- To estimate probabilities, logistic regression is the choice.
- For nonlinear boundaries, kernel SVMs are popular; kernels can be used with logistic regression and LDA as well, but computations are more expensive.

Summary

- A separating hyperplane classifies by $\text{sign} f(x)$; the maximal margin classifier maximises $M = 1/\|\beta\|$ and depends only on support vectors.
- The support vector classifier allows margin violations with slacks under a budget C (chosen by CV).
- The solution depends on the data only through inner products, so kernels (polynomial, radial) give nonlinear SVMs without explicit feature expansion.
- OVA and OVO extend SVMs to $K > 2$ classes.
- SVM = hinge loss + ridge penalty, close to penalised logistic regression; use logistic regression when probabilities are needed.